

8-2 · DAG 이해 및 유즈케이스 연구 · 정리·회고 0.5H

정리·회고

DAG 작성 규칙을 한 장으로, 8-3 설계안을 확정

수도권ICT 이노베이션스퀘어
데이터 엔지니어링 & RAG 파이프라인 160H · 3기

이 시간의 목표

30분 뒤에는

- ① DAG 선언·순서·재시도·CLI 명령을 한 장으로 정리한다
- ② 자주 겪는 오류 네 가지의 원인과 해결을 안다
- ③ pipeline_skeleton.py 와 minio·qdrant Up 으로 8-3 준비를 마친다
- ④ 오늘의 체크리스트를 스스로 확인한다

논리적 날짜·paused·catchup — 오늘 헛갈렸던 것은 여기서 털고 갑니다

DAG 파일 안 — 선언·태스크·순서·재시도

하려는 일	코드
DAG 선언	<code>with DAG(dag_id=, start_date=, schedule=, catchup=False) as dag:</code>
파이썬 태스크	<code>PythonOperator(task_id=, python_callable=)</code>
순서 · 병렬 · 합류	<code>t1 >> t2 · t1 >> [t2, t3] · [t2, t3] >> t4</code>
재시도	<code>default_args={"retries": 2, "retry_delay": timedelta(minutes=1)}</code>

학습 DAG 는 catchup=False 를 반드시 명시 — start_date 가 과거면 소급 실행이 쏟아진다

CLI 다섯 — 단위 실행부터 의존성 보기까지

하려는 일	명령
단위 실행	airflow tasks test <dag> <task> <날짜>
활성화	airflow dags unpause <dag>
수동 실행	airflow dags trigger <dag>
파싱 오류	airflow dags list-import-errors
의존성 보기	airflow dags show <dag>

모두 `docker compose exec airflow-scheduler airflow` 뒤에 붙인다 — CLI 변수로 줄여 두면 편하다

여덟 개를 스스로 확인해 보세요

- 1 DAG·Task·Operator 관계를 그림으로, 나누는 기준을 한 문장으로
- 2 hello_dag 가 웹 UI 목록에 나타나는 것을 확인했다
- 3 구문 오류를 일부러 내 list-import-errors 로 잡아 봤다
- 4 airflow tasks test 로 태스크 단위 실행을 검증했다
- 5 unpause → trigger 로 전체를 돌리고 Graph 화살표와 대조했다
- 6 중간 태스크를 실패시켜 upstream_failed 를 보고 Clear 로 재실행했다
- 7 병렬·합류 DAG 와 순환 거부를 직접 봤다
- 8 멍등 여부를 적은 분해안과 pipeline_skeleton 골격을 만들었다

증상으로 찾는 원인과 해결

증상	원인	해결
저장했는데 목록에 안 나타난다	dags/ 밖에 두었거나 구문 오류·스캔 주기	위치 확인 → list-import-errors
켜자마자 과거 실행이 잔뜩 쏟아진다	start_date 과거 + catchup=True(미지정)	catchup=False 명시 · 쌓인 실행은 Grid 에서 삭제
trigger 는 됐는데 태스크가 계속 회색	DAG 가 paused — 새 DAG 의 기본값	웹 UI 토글 또는 dags unpause <dag_id>
DAG 파일 하나로 전체가 느려진다	최상위에 무거운 코드 — 파싱마다 실행	무거운 일은 태스크 함수 안에

DAG 파일은 스케줄러가 주기적으로 통째로 파싱한다 — 최상위에 DB 접속·API 호출을 두지 않는다

자주 나오는 질문 네 가지

1

태스크끼리 값을 주고받으려면?

XCom — 다만 큰 데이터는 넘기지 말고 저장소 경로만 넘기는 것이 원칙

2

태스크가 몇 개까지 괜찮나요?

수십 개까지는 흔하다 — 수백이 되면 DAG 를 나누거나 동적 생성을 검토

3

schedule=None 이면 자동 실행이 안 되는데 왜 쓰나요?

다른 DAG 가 트리거하거나 사람이 돌리는 작업에 — 학습에도 편하다

4

TaskFlow API(@task)는 안 배우나요?

더 간결한 최신 문법 — 오늘은 Operator 로 구조를 먼저, 여유가 되면 심화 과제로

8-3 준비

다음 차시를 위한 준비 — 세 줄로 끝

- 1** `cd ~/airflow-lab · ls dags/`
pipeline_skeleton.py 가 있어야 한다
- 2** `docker start minio qdrant`
8-3 에서는 이전 주차 컨테이너를 다시 쓴다
- 3** `docker ps` 로 Status 확인
메모리가 빠듯하면 Spark 는 8-3 에서 필요할 때 켜다

준비 완료

골격 파일 있고
둘 다 Up

심화 과제(선택): 크론 표기로 다음 실행 시각 보기 · TaskFlow API 로 다시 쓰기 · XCom 으로 건수 넘기기

다음 회차 예고

8-3 · 수집→Lake→Spark→VectorDB 흐름 자동화

01

이론 1H

파이프라인
자동화 설계

02

실습 1.5H

수집→Lake→변환
태스크 구현

03

실습 1H

임베딩→VectorDB
전체 DAG 완성

04

정리 0.5H

실행 결과 점검
컬렉션 상태 확인

준비: Airflow 기동(docker compose ps) · OpenAI API 키 · 오늘确定的한 태스크 분해안을 옆에 두고 시작

8-2 · 정리·회고 0.5H 마무리

다음 회차는 8-3 흐름 자동화

손으로 돌리던 수집→Lake 적재→변환→임베딩→VectorDB 적재를 DAG 하나로 잇고, 트리거 한 번으로 끝까지 도는 것을 확인합니다

✓ Qdrant 를 Airflow compose 에 붙이기

✓ 태스크 간 데이터는 XCom 아닌 스토리지 경로로

✓ SparkSubmitOperator 의 구조

✓ 재시도 가능한 태스크 단위로 분해

준비할 것

분해안 문서 · 골격 DAG · OpenAI API 키